Maestría en Arquitectura de Software

Modalidad virtual

Dominó con patrones

Documento de diseño del proyecto semestral

Entrega 2: patrones de comportamiento

Presentado por

Julián Felipe Sánchez Tovar

Presentado a

Fabio Espitia

Asignatura Diseño Basado en Patrones (53343002)
Periodo académico 2026-II

Universidad Antonio Nariño

Bogotá D.C., 30 de octubre de 2026

Índice general

Control de versiones . . . . . **VI**

1 Propuesta del juego **1**
1.1 Descripción general 1
1.2 Características del juego 2
1.3 Aspectos susceptibles de variación 2

2 Reglas del juego **3**
2.1 Reglas de negocio implementadas 3
2.2 Reparto y fichas para robar según la configuración 4
2.3 Reglas excluidas del alcance 4

3 Lenguaje y organización de la solución **6**
3.1 Lenguaje declarado 6
3.2 Estructura de la solución 6
3.3 Dependencias externas 7
3.4 Estado del código 7

4 Casos de uso **9**
4.1 Actores del sistema 9
4.2 Inventario de casos de uso 10
4.3 Especificación del caso de uso CU-01 Configurar partida 11

5 Modelo de dominio **14**
5.1 Conceptos del dominio 14
5.2 Valores enumerados 16
5.3 Restricciones del modelo 17

6 Diseño de clases **17**
6.1 Convenciones del diagrama 17
6.2 Clases del dominio 21
6.3 Comportamiento de las operaciones principales 24
6.4 Clases fuera del dominio 25
6.5 Tamaño del diseño 26

7 Aplicación de la orientación a objetos y asignación de responsabilidades . **27**
7.1 Principios de la orientación a objetos 27
7.2 Principios de asignación de responsabilidades 28
7.3 Puntos de variación identificados 30

8 Patrones de diseño **30**

8.1 Tabla de decisión 30
8.2 Ficha del patrón Strategy 34
8.3 Ficha del patrón State 36
8.4 Ficha del patrón Command 38

9 Medición de recursos y tiempo 40
9.1 Acciones críticas declaradas 40
9.2 Estado de la medición 40

10 Bitácora de decisiones 42
10.1 Declaración de uso de asistentes de inteligencia artificial 47

Referencias bibliográficas 48

Índice de figuras

1 Diagrama de casos de uso del juego de dominó 9
2 Modelo de dominio del juego de dominó 14
3 Enumeraciones del modelo de dominio 16
4 Diagrama de clases de diseño: entidades, interfaces y proyectos 19
5 Diagrama de clases de diseño: tipos de valor del dominio 20
6 Strategy: estructura antes y después de aplicarlo 35
7 State: estructura antes y después de aplicarlo 37
8 Diagrama de estados de la partida 37
9 Command: estructura antes y después de aplicarlo 39

Índice de tablas

1 Historial de versiones del documento vi
2 Características del juego que aportan material de diseño 2
3 Reglas de negocio del juego 3
4 Fichas para robar por tipo de dominó y número de jugadores 4
5 Proyectos que conforman la solución 6
6 Comandos para ejecutar la solución 8
7 Inventario de casos de uso del sistema 10
10 Conceptos del modelo de dominio 15
11 Enumeraciones del dominio 16
12 Clases del proyecto `Domino.Dominio` y su responsabilidad 21
13 Tipos de valor del proyecto `Domino.Dominio` 23
14 Variación del número de clases respecto de la Entrega 1 26
15 Aplicación de los principios de la orientación a objetos 27
16 Justificación de las responsabilidades según los principios GRASP 28
17 Problemas, patrones candidatos y decisión tomada 31
18 Ficha del patrón Strategy 34
19 Ficha del patrón State 36
20 Ficha del patrón Command 38
21 Acciones críticas objeto de medición 40
22 Bitácora de decisiones de diseño 42

Control de versiones

La Tabla 1 registra los cambios de cada versión del documento.

Tabla 1
Historial de versiones del documento

Versión	Fecha	Entrega	Descripción del cambio
1.0	02/10/2026	Entrega 1	Versión inicial. Propuesta del juego, reglas de negocio, organización de la solución, casos de uso, modelo de dominio, diagrama de clases, justificación de responsabilidades, declaración del componente de medición y bitácora de decisiones.
2.0	30/10/2026	Entrega 2	Nueva sección de patrones de diseño: tabla de decisión con ocho problemas y fichas de Strategy, State y Command con sus diagramas antes y después, y diagrama de estados de la partida. Diagrama de clases actualizado al código: estrategias de la CPU, estados de la partida, jugadas como comandos y `PantallaConsola`. Se retira el estado Cerrada del modelo y RN-04 precisa que el límite de tiempo no corre en la salida. Estado del código: juego jugable de principio a fin, consola interactiva y proyecto de pruebas. Nuevas entradas en la bitácora y en la declaración de uso de inteligencia artificial.

1. Propuesta del juego

1.1. Descripción general

El sistema es un juego de dominó por turnos, ejecutado en consola, en el que un jugador humano se enfrenta a entre uno y tres jugadores automáticos, en adelante CPU, en la misma máquina. Antes de comenzar, el jugador elige el tipo de dominó (doble 6 o doble 9), la cantidad de CPU, el nivel de dificultad de cada una (Principiante, Intermedio o Avanzado) y, de forma opcional, un límite de tiempo por turno. La partida se juega a una sola mano y siempre de forma individual: gana quien se quede primero sin fichas o, si el juego se cierra, quien conserve la menor suma de puntos.

El dominó concentra variación real en puntos identificables. El tipo de dominó modifica el tamaño del juego de fichas, el reparto y la cantidad de fichas disponibles para robar; los jugadores automáticos deciden su jugada con criterios distintos según su nivel; la mano atraviesa estados en los que se admiten jugadas diferentes; y las jugadas deben poder deshacerse, guardarse y recuperarse. Cada uno de estos puntos constituye una fuerza de diseño.

1.2. Características del juego

La Tabla 2 relaciona las características del dominó que aportan material de diseño con su manifestación en el alcance definido.

Tabla 2
Características del juego que aportan material de diseño

Característica	Manifestación en el dominó
Estados de partida con reglas distintas	La partida pasa por los estados preparada, salida, en curso y terminada. En la salida solo se admite la ficha que determina la regla de apertura; en curso se admite colocar, robar y pasar, y una vez terminada, por dominó o por juego cerrado, ya no se admiten jugadas.
Entidades con comportamiento diferente	El jugador humano y los jugadores CPU de tres niveles deciden su jugada de forma distinta.
Variantes	Tipo de dominó, número de CPU, dificultad de cada CPU y límite de tiempo por turno.
Jugadas registrables y reversibles	Historial de la mano, deshacer la última jugada del humano, guardar y cargar la partida.
Estructura contenedora	Mesa con dos extremos abiertos, fichas para robar y mano de cada jugador.

1.3. Aspectos susceptibles de variación

En el juego varían el tipo de dominó, la forma en que cada jugador decide su jugada, el control de tiempo del turno y el medio en el que se guarda la partida. Adicionalmente, a futuro podrían variar la forma de presentación (consola o interfaz gráfica) y el tipo de jugador, por ejemplo un jugador remoto, sin que estos cambios deban afectar al dominio.

2. Reglas del juego

2.1. Reglas de negocio implementadas

Las reglas de negocio de la Tabla 3 se identifican con un código que se referencia desde los casos de uso y desde el modelo de dominio.

Tabla 3
Reglas de negocio del juego

Código	Regla
RN-01	Los tipos de dominó disponibles son doble 6, con 28 fichas, y doble 9, con 55 fichas.
RN-02	La partida tiene un jugador humano y de una a tres CPU, para un total de dos a cuatro jugadores. Siempre se juega de forma individual.
RN-03	Cada jugador recibe 7 fichas en doble 6 y 10 fichas en doble 9. Las fichas restantes quedan disponibles para robar.
RN-04	El límite de tiempo por turno puede ser sin límite, 10, 20 o 30 segundos, y aplica solo al jugador humano. Si el tiempo se agota, el jugador pierde el turno. El límite no corre durante la salida, porque la ficha de salida es una jugada obligada.
RN-05	Abre la mano el jugador que tenga el doble más alto entre todos los jugadores, buscando en orden descendente desde el doble máximo del tipo de dominó hasta el doble 0. Si ningún jugador tiene dobles, abre quien tenga la ficha de mayor puntaje; en caso de empate, la que tenga el número individual más alto.
RN-06	Una ficha solo puede colocarse en un extremo de la mesa con el que coincida uno de sus valores.
RN-07	Si el jugador no tiene ficha para lanzar, debe robar hasta obtener una ficha jugable o hasta que no queden fichas para robar. Si no quedan fichas para robar y no tiene ficha jugable, pasa.
RN-08	Gana la partida el primer jugador que se queda sin fichas.

Código	Regla
RN-09	El juego se cierra cuando no quedan fichas para robar y ningún jugador tiene ficha para lanzar. En ese caso se revelan las fichas de todos los jugadores para corroborar el resultado, y gana quien tenga la menor suma de puntos en sus fichas; en empate, quien tenga menos fichas; si el empate persiste, la partida termina empatada. Perder el turno por tiempo no cuenta para declarar el juego cerrado.
RN-10	El jugador humano puede deshacer su última jugada. Al hacerlo se revierten también las jugadas de las CPU realizadas después de ella, hasta volver a su turno.
RN-11	La partida en curso puede guardarse y cargarse posteriormente en el mismo estado.

2.2. Reparto y fichas para robar según la configuración

La cantidad de fichas que quedan para robar depende de la combinación entre el tipo de dominó y el número de jugadores, como se resume en la Tabla 4. Con doble 6 y cuatro jugadores se reparten las 28 fichas, de modo que no quedan fichas para robar y la regla RN-07 se reduce a pasar.

Tabla 4
Fichas para robar por tipo de dominó y número de jugadores

Jugadores	Doble 6	Doble 9
	7 fichas c/u	10 fichas c/u
Humano y 1 CPU	14	35
Humano y 2 CPU	7	25
Humano y 3 CPU	Ninguna	15

2.3. Reglas excluidas del alcance

Las siguientes variantes quedan por fuera del proyecto. Las razones de cada exclusión se registran en la bitácora de decisiones (sección 10).

1. Juego en parejas.

2. Multijugador en red.
3. Doble 12 y otros tipos de dominó.
4. Puntos acumulados entre manos y meta de puntos.
5. Deshacer más de una jugada y rehacer.
6. Bonificaciones especiales, como la capicúa.

3. Lenguaje y organización de la solución

3.1. Lenguaje declarado

El proyecto se desarrolla en C# sobre la plataforma .NET 10.

3.2. Estructura de la solución

La solución se divide en proyectos cuyas dependencias apuntan en una sola dirección, hacia el dominio. De este modo la lógica del juego no conoce la consola ni ningún otro medio de entrada y salida. La Tabla 5 describe cada proyecto.

Tabla 5
Proyectos que conforman la solución

Proyecto	Responsabilidad	Depende de
`Domino.Dominio`	Reglas y conceptos del juego: partida, mesa, fichas, fichas para robar, jugadores y turnos.	Ninguno
`Domino.Persistencia`	Guardado y carga de partidas en archivos JSON, detrás de `IRepositorioPartidas`.	`Domino.Dominio`
`Domino.Consola`	Interfaz de texto: presenta el estado del juego y traduce las acciones del jugador.	`Domino.Dominio`, `Domino.Persistencia`
`Domino.Simulacion`	Ejecución de partidas completas sin interfaz, con semilla fija, para la medición.	`Domino.Dominio`, `Domino.Persistencia`
`Domino.Pruebas`	Pruebas automatizadas (xUnit) de las reglas del juego y de los patrones aplicados. No forma parte del diseño.	`Domino.Dominio`, `Domino.Persistencia`

> **Restricción de independencia del dominio.** El proyecto `Domino.Dominio` no referencia a `System.Console` ni a ningún otro proyecto de la solución. La consola puede reemplazarse por otra interfaz sin modificar el dominio, y `Domino.Simulacion` lo evidencia al ejecutar partidas completas utilizando únicamente el dominio.

3.3. Dependencias externas

El control del tiempo por turno y la persistencia de la partida utilizan componentes de la plataforma, pero el dominio los consume a través de abstracciones propias (`IReloj` e `IRepositorioPartidas`). De este modo la simulación emplea un reloj controlado, que no espera segundos reales, y el medio de guardado puede cambiarse sin afectar las reglas. La persistencia reside en un proyecto propio porque la utilizan tanto la consola como la simulación, que mide el guardado y la carga.

3.4. Estado del código

El repositorio (`https://github.com/DevJulianSanchez/domino-con-patrones`), con la etiqueta `E2`, contiene en la carpeta `Documentación` este documento con sus diagramas y, en la carpeta `Código`, la solución `Domino.slnx`, que compila sin advertencias. Las clases del código tienen los mismos nombres y miembros que el diagrama de clases (sección 6).

En esta entrega el juego se puede jugar de principio a fin. La partida se configura (CU-01), reparte y determina la salida (RN-05), y admite colocar, robar y pasar (RN-06 y RN-07), el límite de tiempo por turno (RN-04), el dominó (RN-08), el juego cerrado con revelación de fichas (RN-09), deshacer (RN-10) y guardar y cargar (RN-11). Los tres niveles de CPU deciden con criterios distintos. Se aplicaron los patrones de comportamiento Strategy, State y Command, documentados en la sección 8.

La consola dibuja un único tablero que se actualiza en su lugar, con los puntos de cada ficha, y se maneja con el teclado. Su reemplazo, desde la versión por líneas de la Entrega 1, no modificó ninguna clase del dominio. `Domino.Simulacion` ejecuta partidas completas con semilla fija y ejercita en cada una deshacer, guardar y cargar. El proyecto `Domino.Pruebas` contiene 25 pruebas que verifican las reglas y el comportamiento de los tres patrones.

Tabla 6
Comandos para ejecutar la solución

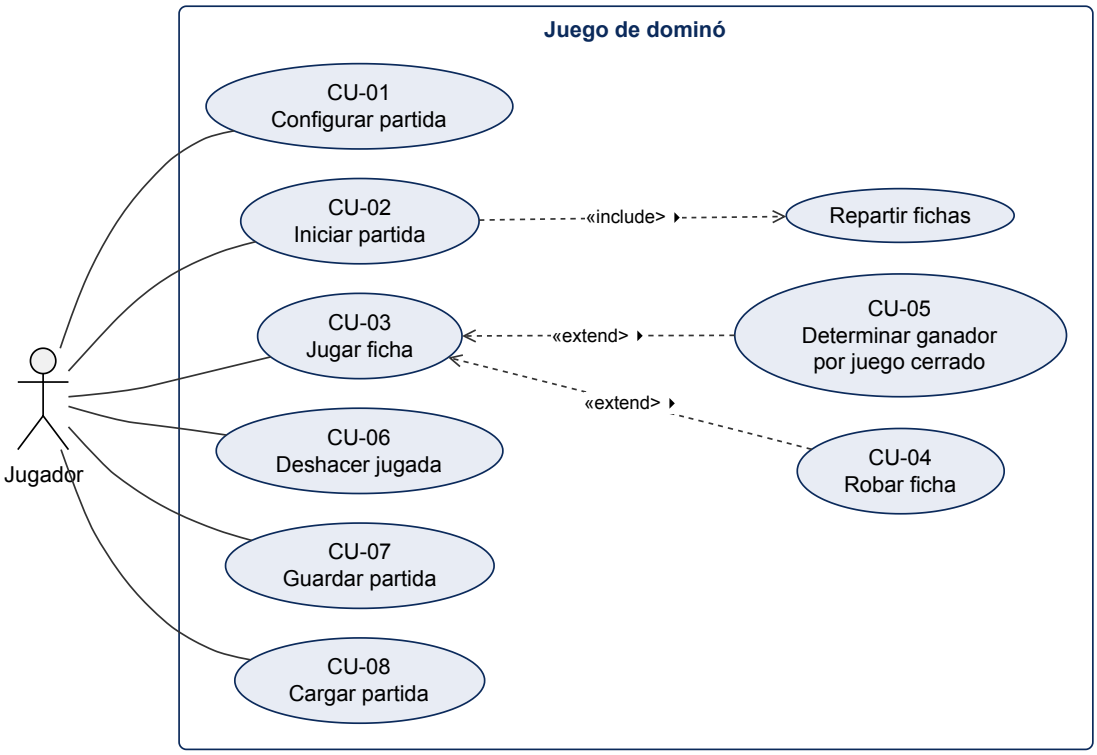
Comando (desde la carpeta `Código`)	Resultado
`dotnet run --project src/Domino.Consola`	Juego en consola.
`dotnet run --project src/Domino.Simulacion -- 1000 42`	1000 partidas sin interfaz con semilla 42; informa las partidas ganadas por cada jugador.
`dotnet test`	Ejecuta las pruebas automatizadas.

4. Casos de uso

4.1. Actores del sistema

El único actor del sistema es el jugador humano. Los jugadores CPU no se modelan como actores, dado que no son externos al sistema: su comportamiento forma parte del propio juego y se manifiesta dentro de los casos de uso, por ejemplo cuando el sistema resuelve los turnos de las CPU antes de devolver el turno al jugador (**Larman, 2003**).

Figura 1
Diagrama de casos de uso del juego de dominó



4.2. Inventario de casos de uso

La Tabla 7 relaciona los casos de uso identificados, sus relaciones y las reglas de negocio que los rigen.

Tabla 7
Inventario de casos de uso del sistema

Código	Caso de uso	Relación y observaciones
CU-01	Configurar partida	Especificado en detalle a continuación.
CU-02	Iniciar partida	Incluye Repartir fichas y aplica la regla de salida (RN-05).
CU-03	Jugar ficha	Caso central. El tiempo agotado se trata como curso alterno, en el que el jugador pierde el turno (RN-04).
CU-04	Robar ficha	Extiende a CU-03 cuando el jugador no tiene ficha para lanzar y quedan fichas para robar (RN-07).
CU-05	Determinar ganador por juego cerrado	Extiende a CU-03 cuando no quedan fichas para robar y ningún jugador tiene ficha para lanzar. Se revelan las fichas de todos los jugadores y gana la menor suma de puntos (RN-09).
CU-06	Deshacer jugada	Revierte la última jugada del humano y las de las CPU posteriores (RN-10).
CU-07	Guardar partida	Rige la regla RN-11.
CU-08	Cargar partida	Rige la regla RN-11.

La numeración de los cursos alternos sigue la convención propuesta por Cockburn (2001): el número indica el paso del curso normal en el que se detecta la condición, la letra distingue alternativas de un mismo paso y el asterisco identifica las que pueden ocurrir en cualquier paso.

4.3. Especificación del caso de uso CU-01 Configurar partida

Encabezado

Caso de uso	**CU-01. Configurar partida**
Actores	Jugador
Tipo	Primario y esencial
Requisitos	RF-01 Seleccionar el tipo de dominó. RF-02 Definir el número y el nivel de dificultad de las CPU. RF-03 Definir el límite de tiempo por turno.
Casos relacionados	No incluye ni extiende otros casos de uso. CU-02 Iniciar partida requiere la configuración registrada por este caso de uso.
Precondición	No hay una partida en curso en el sistema.
Postcondición	Existe una partida en estado Preparada con el tipo de dominó elegido, el jugador humano identificado por su alias, entre una y tres CPU con su nivel de dificultad asignado y el límite de tiempo por turno definido.
Autor	Julián Felipe Sánchez Tovar
Fecha y versión	27/09/2026 · versión 1.0

Propósito

Permitir que el jugador defina las condiciones de una partida local contra jugadores CPU antes de comenzarla.

Resumen

El jugador solicita una nueva partida, indica su alias y elige el tipo de dominó entre doble 6 y doble 9. Luego define cuántas CPU participan, de una a tres, asigna a cada una un nivel de dificultad y elige el límite de tiempo por turno. El sistema crea la partida en estado Preparada y presenta un resumen con los jugadores, las fichas por jugador y la cantidad de fichas para robar.

Curso normal

Paso	Acción del actor	Paso	Respuesta del sistema
1	El jugador solicita una nueva partida e indica su alias.	2	El sistema valida el alias y presenta las opciones de configuración con sus valores predeterminados: tipo de dominó, número de CPU, nivel de dificultad de cada CPU y límite de tiempo por turno.
3	El jugador selecciona el tipo de dominó y el número de CPU.	4	El sistema presenta cada CPU con el nivel de dificultad predeterminado.
5	El jugador asigna un nivel de dificultad a cada CPU (Principiante, Intermedio o Avanzado), elige el límite de tiempo por turno y confirma.	6	El sistema crea la partida en estado Preparada, con el jugador humano y las CPU configuradas.
		7	El sistema presenta el resumen: tipo de dominó, jugadores con su nivel, fichas por jugador, cantidad de fichas para robar y límite de tiempo.

Cursos alternos

Ref.	Condición, acciones alternas y cierre
2a	El alias está vacío o supera la longitud permitida: el sistema informa el motivo y solicita un nuevo alias. Vuelve al paso 1.
3a	El jugador confirma sin modificar los valores predeterminados: el sistema toma la configuración predeterminada. Continúa en el paso 6.
***a**	El jugador cancela antes del paso 6: no se crea la partida. El caso de uso termina.

Otros datos

Frecuencia	Una vez por partida	**Rendimiento**	Creación de la partida en menos de 1 segundo desde la confirmación
Importancia	Vital	**Urgencia**	Alta
Estado	En elaboración	**Estabilidad**	Moderada

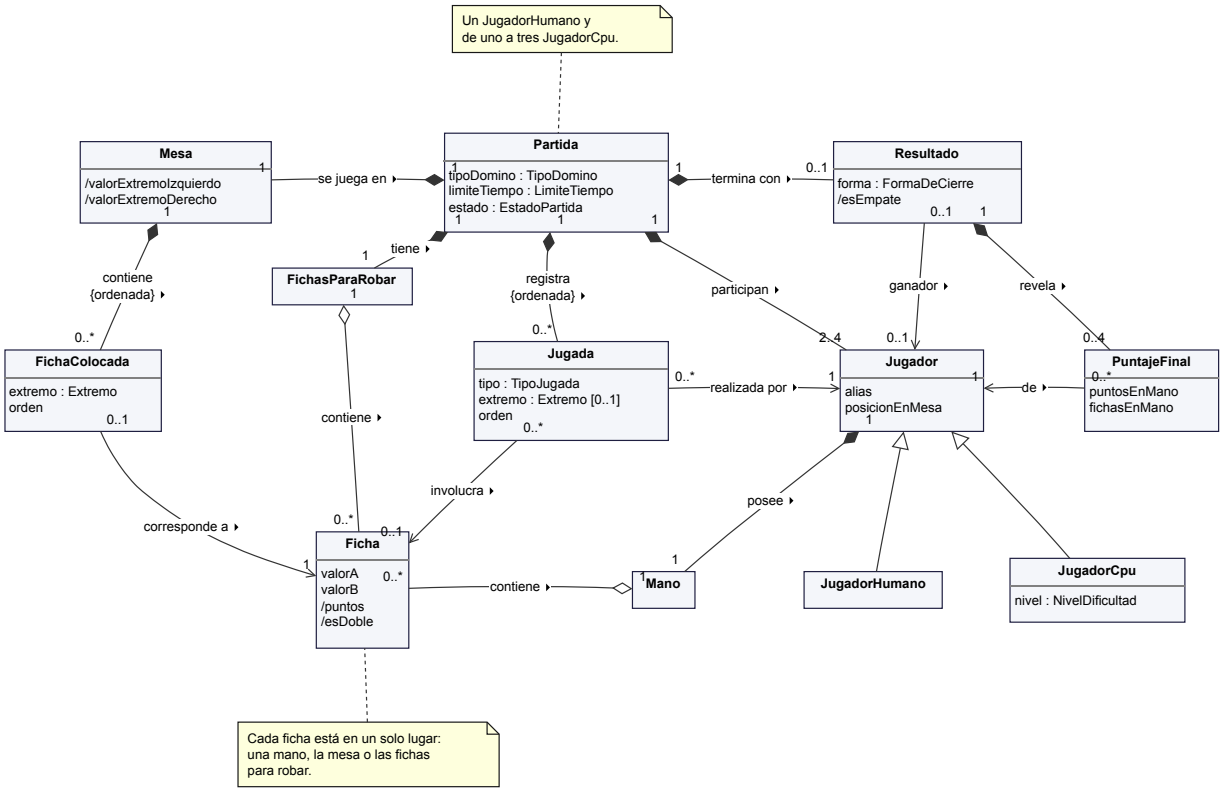
Comentarios

Reglas de negocio aplicadas. RN-01, RN-02, RN-03 y RN-04.
Supuestos. Los valores predeterminados son doble 6, una CPU de nivel Intermedio y sin límite de tiempo. Todas las opciones se eligen de listas cerradas, por lo que ninguna combinación resulta inválida y no se requiere un paso de validación de la configuración.

5. Modelo de dominio

El modelo de dominio representa los conceptos del juego tal como los entiende un jugador, con sus atributos y asociaciones, sin operaciones ni decisiones de tecnología (**Larman, 2003**). Los conceptos se obtuvieron de las reglas de negocio (sección 2) y de los casos de uso (sección 4). La consola, el reloj del sistema y el almacenamiento en disco no forman parte del modelo, porque no son conceptos del juego sino medios técnicos.

Figura 2
Modelo de dominio del juego de dominó



5.1. Conceptos del dominio

La Tabla 10 describe cada concepto y la regla de negocio que lo origina. Los atributos precedidos de una barra en el diagrama son derivados: se calculan a partir de otros datos y no se registran por separado.

Tabla 10
Conceptos del modelo de dominio

Concepto	Descripción	Origen
`Partida`	Encuentro configurado entre un jugador humano y de una a tres CPU. Registra el tipo de dominó, el límite de tiempo por turno y el estado.	CU-01, RN-01, RN-02, RN-04
`Jugador`	Participante de la partida, identificado por su alias y su posición en la mesa. Se especializa en `JugadorHumano` y `JugadorCpu`.	RN-02
`JugadorCpu`	Jugador automático con un nivel de dificultad: Principiante, Intermedio o Avanzado.	RN-02
`Mano`	Fichas que posee un jugador y que los demás no ven mientras la partida está en curso.	RN-03, RN-09
`Ficha`	Pieza con dos valores. Sus puntos y su condición de doble se derivan de esos valores.	RN-01, RN-05, RN-09
`Mesa`	Cadena de fichas colocadas, con dos extremos abiertos cuyos valores se derivan de las fichas de los extremos.	RN-06
`FichaColocada`	Ficha puesta en la mesa, con el extremo en el que se colocó y su orden.	RN-06
`FichasParaRobar`	Fichas que quedan después del reparto y de las que roba el jugador sin ficha para lanzar.	RN-03, RN-07
`Jugada`	Registro de cada acción del juego: colocar ficha, robar, pasar o perder el turno por tiempo. Conserva el orden en que ocurrieron.	RN-04, RN-10, RN-11
`Resultado`	Forma en que terminó la partida (dominó o juego cerrado) y el ganador, si lo hay.	RN-08, RN-09
`PuntajeFinal`	Puntos y cantidad de fichas que revela cada jugador cuando el juego se cierra.	RN-09

5.2. Valores enumerados

Los atributos con un conjunto cerrado de valores se representan como enumeraciones, de modo que las restricciones de la configuración y del estado quedan expresadas en el propio modelo.

Figura 3
Enumeraciones del modelo de dominio



Tabla 11
Enumeraciones del dominio

Enumeración	Valores
`TipoDomino`	Doble6, Doble9
`NivelDificultad`	Principiante, Intermedio, Avanzado
`LimiteTiempo`	SinLimite, Segundos10, Segundos20, Segundos30
`EstadoPartida`	Preparada, Salida, EnCurso, Terminada
`Extremo`	Izquierdo, Derecho
`TipoJugada`	ColocarFicha, Robar, Pasar, TiempoAgotado
`FormaDeCierre`	Domino, JuegoCerrado

5.3. Restricciones del modelo

- Cada ficha se encuentra en un solo lugar a la vez: la mano de un jugador, la mesa o las fichas para robar.
- Una partida tiene exactamente un `JugadorHumano` y de uno a tres `JugadorCpu`.
- Cuando el juego se cierra, la partida pasa directamente a Terminada: se revelan las fichas de todos los jugadores y el resultado registra la forma de cierre `JuegoCerrado`. En la versión 1.0 el cierre se modelaba como un estado Cerrada previo, que se retiró porque no tenía reglas propias (sección 10).
- Una partida terminada por dominó tiene ganador; una partida cerrada puede terminar empatada, en cuyo caso el resultado no tiene ganador.

6. Diseño de clases

El diagrama de clases de diseño traslada los conceptos del dominio a clases de software con atributos, operaciones, constructores y visibilidad. Es la base directa de la implementación: los nombres, las firmas y los tipos coinciden con los del código del repositorio. Cada clase del dominio indica como estereotipo el principio de asignación de responsabilidades que justifica su existencia o su papel principal (**Larman, 2003**) y, cuando participa en un patrón, el rol que cumple en él (**Gamma et al., 2003**).

6.1. Convenciones del diagrama

- **Entidades y tipos de valor.** Las clases con identidad y estado mutable (partida, mesa, jugadores, mano, historial, estados, estrategias y jugadas) se relacionan mediante asociaciones. Las clases inmutables que se comparan por valor llevan el estereotipo «dataType», se implementan como `record` y se usan como tipo de atributo o de parámetro; por legibilidad se presentan en un diagrama aparte (Figura 5).
- **Extremos de asociación.** El nombre de rol corresponde al campo o propiedad que implementa la asociación, con su visibilidad; la multiplicidad se indica en ambos

extremos. Una multiplicidad `0..*` se implementa como colección y {ordered} indica que el orden es significativo.

- **Miembros.** Los miembros abstractos se muestran en cursiva y los estáticos subrayados. La visibilidad `~` (paquete) corresponde a `internal` de C#: el miembro solo es accesible dentro de `Domino.Dominio`. Las subclases muestran los miembros que redefinen o implementan. Los tipos de valor posicionales no muestran su constructor, porque coincide con sus atributos.
- **Roles de patrón.** Los estereotipos *Strategy*, *ConcreteStrategy*, *State*, *ConcreteState*, *Command*, *ConcreteCommand*, *Context*, *Invoker* y *Receiver* indican el papel de la clase en los patrones de la sección 8.

Figura 4

Diagrama de clases de diseño: entidades, interfaces y proyectos

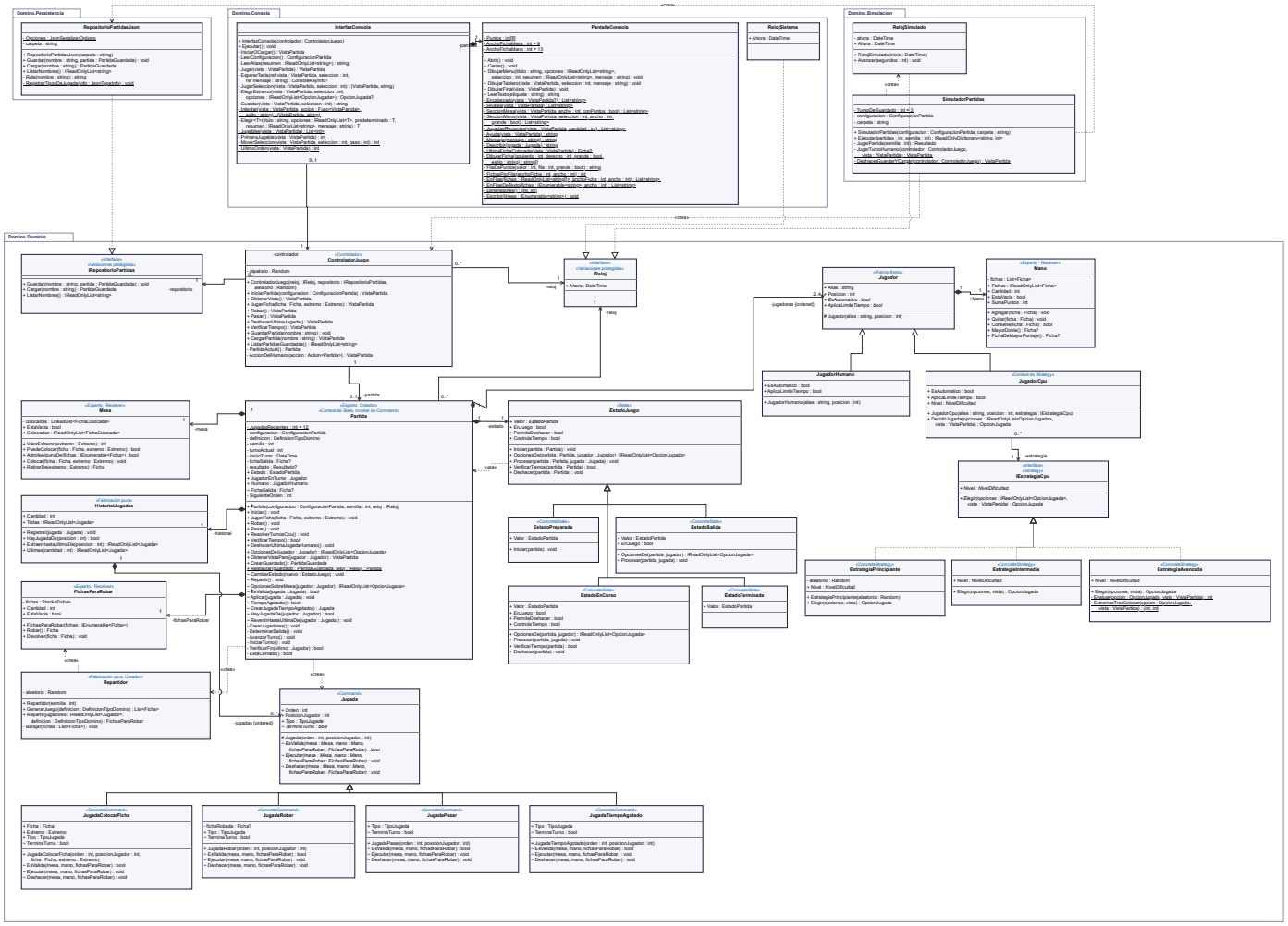
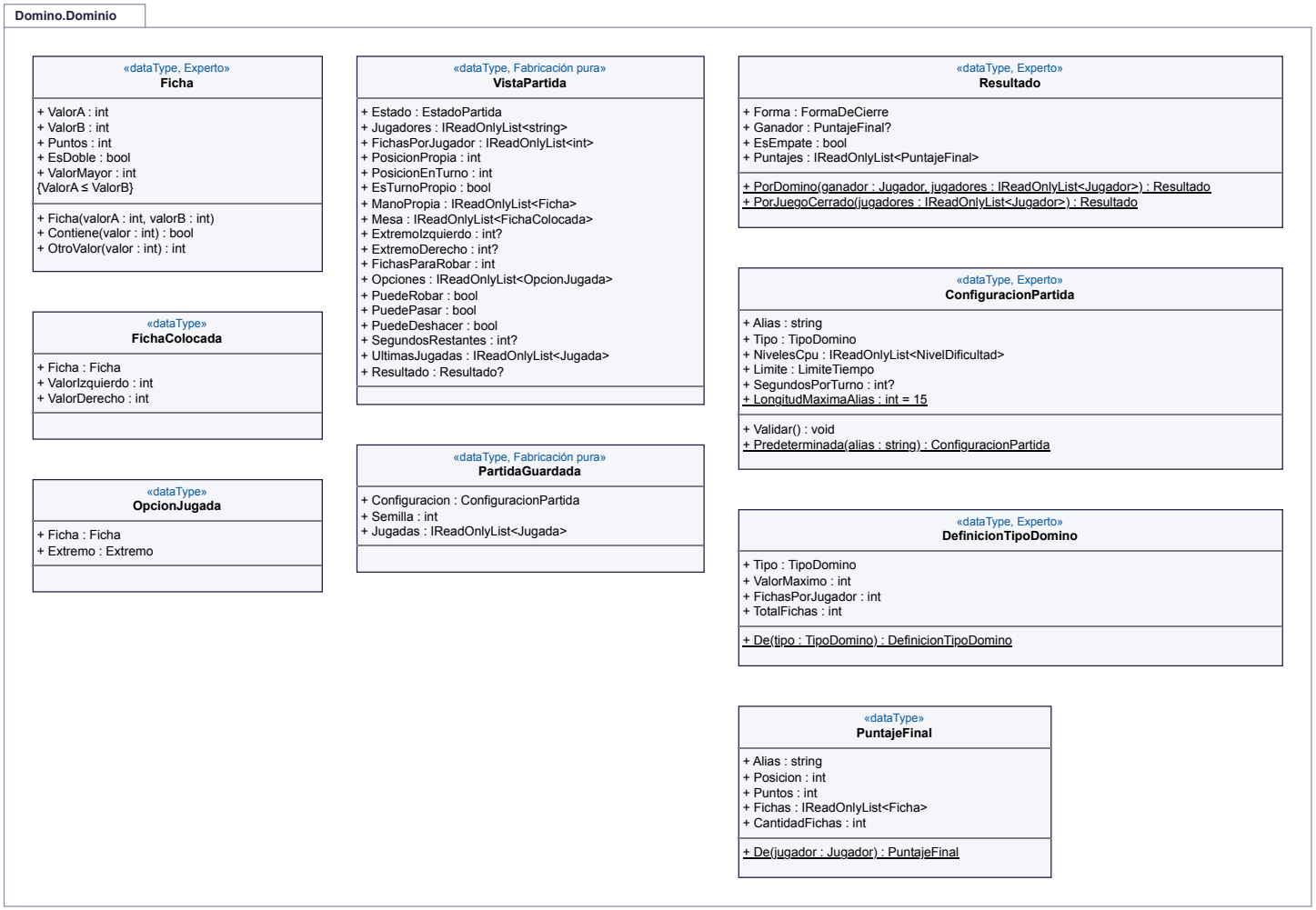


Figura 5

Diagrama de clases de diseño: tipos de valor del dominio



6.2. Clases del dominio

Tabla 12
Clases del proyecto `Domino.Dominio` y su responsabilidad

Clase	Responsabilidad
`ControladorJuego`	Recibe las operaciones del sistema que provienen de la interfaz, las delega en la partida, resuelve a continuación los turnos de las CPU y devuelve la vista del jugador humano. Genera la semilla de cada partida y coordina guardar y cargar.
`Partida`	Aplica las reglas del juego y coordina mesa, jugadores, reparto e historial. Delega en su estado actual las acciones que dependen de él (Context de State), crea las jugadas, las ejecuta, las registra y las revierte (Invoker de Command), y crea los jugadores con la estrategia de su nivel.
`Mesa`	Mantiene la cadena de fichas colocadas, valida si una ficha puede colocarse en un extremo, la orienta al colocarla y permite retirarla al deshacer.
`FichasParaRobar`	Entrega fichas al jugador que roba y las recibe de vuelta al deshacer, en orden inverso.
`HistorialJugadas`	Registra cada jugada en orden y extrae la secuencia que debe revertirse al deshacer.
`Repartidor`	Genera el juego de fichas del tipo de dominó, lo baraja con una semilla y lo reparte; entrega las fichas restantes como fichas para robar.
`Jugador`	Clase abstracta con el alias, la posición y la mano. Declara si el jugador es automático y si le aplica el límite de tiempo.
`JugadorHumano`	Jugador cuyas decisiones llegan desde la interfaz; le aplica el límite de tiempo.
`JugadorCpu`	Jugador automático: no le aplica el límite de tiempo y delega la decisión de jugada en su estrategia, que define su nivel.

Clase	Responsabilidad
`Mano`	Administra las fichas de un jugador y calcula su suma de puntos, su mayor doble y su ficha de mayor puntaje.
`IEstrategiaCpu`	Criterio con el que una CPU elige entre las opciones válidas.
`EstrategiaPrincipiante`	Elige al azar una de las opciones válidas.
`EstrategiaIntermedia`	Elige la opción cuya ficha suma más puntos, para descargar la mano; en empate prefiere los dobles.
`EstrategiaAvanzada`	Evalúa cada opción por el número de fichas propias que seguirían siendo jugables con los extremos resultantes y desempata por puntos.
`EstadoJuego`	Clase abstracta con el comportamiento que depende del estado: jugadas admitidas, opciones, control del tiempo y deshacer. Por omisión no admite ninguna acción.
`EstadoPreparada`	Solo admite iniciar: reparte y pasa a Salida.
`EstadoSalida`	Solo admite la ficha de salida, sin límite de tiempo, y pasa a EnCurso.
`EstadoEnCurso`	Admite colocar, robar y pasar según las reglas, controla el tiempo del turno y permite deshacer.
`EstadoTerminada`	No admite acciones; la vista muestra el resultado.
`Jugada`	Clase abstracta de las acciones del juego: valida, ejecuta y deshace la acción sobre la mesa, la mano y las fichas para robar, e indica si termina el turno.
`JugadaColocarFicha`	Quita la ficha de la mano y la coloca en el extremo indicado; al deshacer, la retira de la mesa y la devuelve a la mano.
`JugadaRobar`	Roba una ficha y recuerda cuál fue para devolverla al deshacer. No termina el turno.
`JugadaPasar`	Pasa el turno cuando no hay ficha para lanzar ni fichas para robar.
`JugadaTiempoAgotado`	Registra la pérdida del turno por tiempo.

Clase	Responsabilidad
`IReloj`, `IRepositorioPartidas`	Abstracciones propias para la hora actual y para guardar, cargar y listar partidas guardadas.

Tabla 13
Tipos de valor del proyecto `Domino.Dominio`

Tipo	Responsabilidad
`Ficha`	Par de valores normalizado (`ValorA` $\leq$ `ValorB`), de modo que dos fichas iguales son iguales sin importar su orientación. Calcula sus puntos, si es doble y si contiene un valor.
`FichaColocada`	Ficha orientada en la mesa, con el valor que queda hacia cada lado.
`OpcionJugada`	Par ficha y extremo que representa una jugada válida disponible.
`PuntajeFinal`	Alias, puntos y fichas reveladas de un jugador al terminar la partida.
`Resultado`	Forma de cierre, puntajes de todos los jugadores y ganador; aplica el desempate de RN-09.
`ConfiguracionPartida`	Datos de CU-01 con su validación y los valores predeterminados; traduce el límite de tiempo a segundos.
`DefinicionTipoDomino`	Valor máximo, fichas por jugador y total de fichas de cada tipo de dominó.
`VistaPartida`	Información visible para un jugador: su mano, la cantidad de fichas de los demás, la mesa, las opciones válidas, las acciones permitidas, el tiempo restante, las últimas jugadas y, al terminar, el resultado.
`PartidaGuardada`	Configuración, semilla y jugadas de una partida: lo necesario para reconstruirla.

En la versión 1.0, `Jugada` era un tipo de valor. Al aplicar Command pasó a ser una jerarquía de clases con comportamiento (sección 8). Sus operaciones de ejecución son de paquete, de modo que la interfaz, que recibe jugadas dentro de `VistaPartida`, puede leerlas pero no ejecutarlas.

6.3. Comportamiento de las operaciones principales

- **Iniciar.** `Partida.Iniciar` delega en el estado. `EstadoPreparada` pide a la partida que reparta con el `Repartidor` y la semilla, determine quién abre y con qué ficha (RN-05) y pase a `EstadoSalida`. Si la salida corresponde a una CPU, `ControladorJuego` la juega de inmediato.
- **Jugadas.** `JugarFicha`, `Robar` y `Pasar` crean la jugada correspondiente y la entregan a `Procesar` del estado actual. `EstadoSalida` solo acepta la ficha de salida y pasa a `EstadoEnCurso`. `EstadoEnCurso` acepta la jugada si corresponde al jugador en turno y cumple sus propias condiciones (`EsValida`). En ambos casos, `Partida.Aplicar` ejecuta la jugada, la registra, verifica el fin y, si la jugada termina el turno, lo avanza. Robar no termina el turno: el jugador sigue robando hasta tener ficha para lanzar o agotar las fichas para robar (RN-07).
- **Turnos de las CPU.** `ResolverTurnosCpu` se ejecuta mientras el estado admita jugadas y el turno sea de una CPU: si hay opciones, la CPU decide con su estrategia sobre su propia vista; si no, roba o pasa.
- **Tiempo.** Cada cambio de turno registra `inicioTurno` con `IReloj`. Solo `EstadoEnCurso` controla el tiempo: si el jugador en turno tiene límite y se agotó, aplica una `JugadaTiempoAgotado` (RN-04). El controlador lo verifica antes de cada acción del humano, y la consola lo consulta periódicamente para que el turno se pierda en el momento en que se agota.
- **Fin.** `VerificarFin` declara el dominó cuando el jugador queda sin fichas (RN-08) y el juego cerrado cuando no quedan fichas para robar y ningún jugador tiene ficha para lanzar (RN-09); en ambos casos construye el `Resultado` y pasa a `EstadoTerminada`.

- **Deshacer.** Solo `EstadoEnCurso` lo admite. `HistorialJugadas.ExtraerHastaUltimaDe` entrega, de la más reciente a la más antigua, las jugadas desde la última del humano, y cada una se deshace sobre sus receptores. El turno vuelve al humano y, si la mesa quedó vacía, la partida regresa a `EstadoSalida` (RN-10).
- **Guardar y cargar.** `CrearGuardado` produce la configuración, la semilla y la lista de jugadas. `Restaurar` crea una partida con la misma configuración y semilla, la inicia, lo que reproduce el mismo reparto, y vuelve a procesar las jugadas guardadas en orden a través del estado, sin consultar a las CPU (RN-11).

6.4. Clases fuera del dominio

Las siguientes clases residen en los proyectos que dependen del dominio. Ninguna de ellas es referenciada por `Domino.Dominio`: se relacionan con él a través de `ControladorJuego` y de las interfaces que implementan. Cada proyecto ejecutable arma sus dependencias en su punto de entrada.

- `Domino.Persistencia`: `RepositorioPartidasJson`, que guarda cada `PartidaGuardada` como un archivo JSON en una carpeta. Registra los cuatro tipos de jugada con un discriminador, de modo que el dominio no depende del formato del archivo.
- `Domino.Consola`: `InterfazConsola`, que lee las teclas y traduce cada acción en una operación del controlador; `PantallaConsola`, que dibuja el tablero completo en una sola pantalla con los puntos de cada ficha; y `RelojSistema`.
- `Domino.Simulacion`: `SimuladorPartidas` y `RelojSimulado`. El simulador ocupa el lugar del jugador humano, juega la primera opción válida, o roba o pasa cuando no la tiene, y en cada partida deshace, guarda y carga una vez.

6.5. Tamaño del diseño

El diseño suma 35 clases e interfaces en el dominio (26 entidades, interfaces y participantes de patrones, y 9 tipos de valor) y 6 en los demás proyectos, para un total de 41, sin contar las siete enumeraciones ni el proyecto de pruebas. La Tabla 14 muestra de dónde provienen las 11 clases agregadas respecto de la Entrega 1.

Tabla 14
Variación del número de clases respecto de la Entrega 1

Cambio	Clases	Detalle
Entrega 1	30	Punto de partida.
Strategy	+1	Se agregan `IEstrategiaCpu` y tres estrategias, y se retiran las tres subclases de `JugadorCpu`.
State	+5	`EstadoJuego` y sus cuatro estados concretos.
Command	+4	`Jugada` y sus cuatro jugadas concretas reemplazan el tipo de valor `Jugada`.
Consola	+1	`PantallaConsola` separa el dibujo del tablero de la lectura de teclas.
Entrega 2	41	

El total supera el rango orientativo de 15 a 30 clases de la especificación. Cada clase adicional cumple un rol en un patrón justificado en la sección 8, y los participantes concretos son clases pequeñas: los estados y las jugadas concretas tienen entre una y cinco operaciones.

7. Aplicación de la orientación a objetos y asignación de responsabilidades

Esta sección justifica el diseño de clases desde dos perspectivas complementarias: los principios de la programación orientada a objetos y los principios de asignación de responsabilidades propuestos por Larman (2003). Para cada decisión relevante se indica la alternativa descartada.

7.1. Principios de la orientación a objetos

Tabla 15
Aplicación de los principios de la orientación a objetos

Principio	Aplicación en el diseño
Abstracción	`Jugador`, `EstadoJuego` y `Jugada` son abstractas e `IEstrategiaCpu` es una interfaz: definen lo que ofrece cada familia sin fijar cómo lo hace. `IReloj` e `IRepositorioPartidas` abstraen la hora y el almacenamiento.
Encapsulamiento	`Mano`, `Mesa`, `FichasParaRobar` y `HistorialJugadas` ocultan sus colecciones internas y solo permiten modificarlas mediante operaciones que preservan las reglas. Por ejemplo, ninguna clase externa puede agregar una ficha a la mesa sin pasar por `PuedeColocar`. `Ficha` es inmutable.
Herencia	`JugadorHumano` y `JugadorCpu` heredan de `Jugador`; los cuatro estados heredan de `EstadoJuego` y las cuatro jugadas de `Jugada`. Los niveles de CPU dejaron de ser subclases: se componen como estrategias (sección 8).
Polimorfismo	`Partida` consulta `AplicaLimiteTiempo` sin distinguir el tipo concreto de jugador, delega en su estado sin conocer cuál es y ejecuta o deshace jugadas sin conocer su tipo. `JugadorCpu` invoca `Elegir` sin conocer el nivel de su estrategia.

7.2. Principios de asignación de responsabilidades

Tabla 16
Justificación de las responsabilidades según los principios GRASP

Principio	Decisión	Justificación y alternativa descartada
Controlador	`ControladorJuego` recibe todas las operaciones del sistema.	Es el punto único de entrada desde la interfaz, de modo que la consola no conoce `Partida` por dentro. Se descartó que la consola invocara directamente a `Partida`, porque acoplaría la interfaz a las reglas y obligaría a repetir la coordinación en la simulación.
Experto en información	`Ficha` calcula sus puntos y si es doble; `Mano` suma sus puntos y encuentra su mayor doble; `Mesa` decide si una ficha puede colocarse.	Cada responsabilidad queda en la clase que posee los datos necesarios. Se descartó concentrar estos cálculos en `Partida`, que tendría que recorrer las colecciones de otras clases y conocer su estructura interna.
Experto en información	`Resultado` determina el ganador y el desempate al cerrarse el juego (RN-09).	El resultado conoce los puntajes revelados de todos los jugadores. Se descartó ubicar el desempate en `ControladorJuego`, que no posee esa información.
Creador	`Partida` crea `Mesa`, `HistorialJugadas` y los jugadores.	La partida los contiene por composición y dispone de los datos de la configuración para inicializarlos.
Creador y fabricación pura	`Repartidor` crea las fichas y las fichas para robar.	Barajar y repartir no es responsabilidad natural de ningún concepto del juego. Una clase dedicada conserva la cohesión de `Partida` y permite fijar la semilla del barajado en la simulación.
Fabricación pura	`HistorialJugadas` administra la secuencia de jugadas para deshacer.	Separa de `Partida` la lógica de localizar la última jugada del humano y las jugadas de CPU posteriores (RN-10).

Principio	Decisión	Justificación y alternativa descartada
Fabricación pura	`VistaPartida` expone solo la información visible.	Controla en un único lugar qué ve cada jugador, incluida la revelación de fichas al cerrarse el juego. Solo contiene tipos de valor, de modo que la interfaz no puede modificar el dominio a través de ella. Se descartó entregar `Partida` a la interfaz y a las CPU, porque expondría las manos ajenas.
Fabricación pura	`PartidaGuardada` contiene la configuración, la semilla y las jugadas.	Permite guardar la partida sin exponer el estado interno de `Partida`: al cargarla, la partida se reconstruye repitiendo el reparto y las jugadas. Se descartó serializar el objeto `Partida` completo, porque obligaría a hacer públicos sus campos y acoplaría el formato del archivo a su estructura interna.
Polimorfismo	Jerarquías de jugadores, estrategias, estados y jugadas.	El comportamiento que varía según el tipo de jugador, el nivel, el estado de la partida o el tipo de jugada se resuelve con operaciones polimórficas. Se descartaron atributos de tipo con condicionales en `Partida`, que crecerían con cada nuevo caso. Los patrones de la sección 8 son la forma concreta de este principio.
Variaciones protegidas e indirección	`IReloj` e `IRepositorioPartidas`.	Protegen al dominio de los cambios en la fuente de la hora y en el medio de almacenamiento. Permiten que la simulación use un reloj controlado. Se descartó usar directamente `DateTime.Now` y el sistema de archivos desde el dominio.
Bajo acoplamiento	La consola depende solo de `ControladorJuego` y `VistaPartida`; el dominio no depende de ningún otro proyecto.	Cumple la restricción de independencia del dominio y permite reemplazar la interfaz sin modificar las reglas.

Principio	Decisión	Justificación y alternativa descartada
Alta cohesión	Cada clase concentra un único tipo de responsabilidad.	`Partida` coordina las reglas, mientras que el reparto, el historial, la vista y el cálculo de resultados residen en clases propias.

7.3. Puntos de variación identificados

La versión 1.0 dejó identificados cuatro puntos donde el juego varía: la forma en que cada nivel de CPU decide su jugada, el comportamiento de la partida según su estado, la reversión de jugadas y la construcción de la partida a partir de la configuración. Los tres primeros se resolvieron en esta entrega con Strategy, State y Command (sección 8). La construcción de la partida y de sus jugadores corresponde a los patrones creacionales de la Entrega 3.

8. Patrones de diseño

Esta sección registra los problemas del juego en los que se evaluó aplicar un patrón, la decisión tomada en cada uno y, para los patrones aplicados, su ficha. Un patrón se aplicó solo cuando existía una fuerza concreta que lo justificara; en dos de los problemas analizados la decisión fue no aplicar ninguno. Los patrones de estructura y creacionales se identifican aquí como candidatos y se aplican en la Entrega 3 (**Gamma et al., 2003**).

8.1. Tabla de decisión

Tabla 17
Problemas, patrones candidatos y decisión tomada

N.º	Problema y fuerza	Candidatos	Decisión	Por qué se descartaron los otros
D1	Cada nivel de CPU elige su jugada con un criterio distinto, y se espera agregar niveles sin tocar la partida.	Strategy; Template Method; subclases de `JugadorCpu` (diseño 1.0); condicional por nivel.	**Strategy** (ficha en la sección 8.2).	Template Method supone un esqueleto común con pasos variables, pero cada nivel reemplaza la decisión completa. Las subclases atan el criterio al tipo de jugador y obligan a crear una subclase por nivel. Un condicional concentra en una clase criterios que no se relacionan.
D2	Las acciones admitidas cambian según el estado de la partida: preparada, salida, en curso o terminada.	State; condicionales sobre `EstadoPartida` (diseño 1.0); Chain of Responsibility.	**State** (ficha en la sección 8.3).	Los condicionales obligan a consultar el mismo atributo en cada una de las siete operaciones de `Partida` que dependen del estado, y un estado nuevo obliga a revisarlas todas. Chain of Responsibility encadena validaciones, pero aquí la regla depende de cuál estado está activo, no de una secuencia de comprobaciones.
D3	Las jugadas deben poder deshacerse (RN-10) y la partida debe poder guardarse y reconstruirse (RN-11).	Command; Memento; Prototype.	**Command** (ficha en la sección 8.4).	Memento y Prototype guardan instantáneas del estado: resuelven deshacer, pero el archivo guardado expondría el estado interno de la partida y no conservaría qué se jugó, que la vista necesita mostrar. Command usa la misma secuencia de jugadas para deshacer, para mostrar las últimas jugadas y para reconstruir la partida al cargar.

N.º	Problema y fuerza	Candidatos	Decisión	Por qué se descartaron los otros
D4	Validar una jugada antes de ejecutarla: turno, estado, ficha en la mano, coincidencia con el extremo y robo permitido.	Chain of Responsibility; validación en el estado y en la jugada.	**Sin patrón adicional.**	Son cuatro comprobaciones fijas y baratas, sin variación que aislar; una cadena de manejadores agregaría clases sin una fuerza que las justifique. El estado decide qué jugadas admite y cada jugada verifica sus propias condiciones con `EsValida`.
D5	Informar a la interfaz las jugadas que hicieron las CPU.	Observer; consulta de la vista después de cada operación.	**Sin patrón adicional.**	Hay un único interesado, la interfaz o el simulador, que recibe la vista como respuesta de cada operación, y `VistaPartida.UltimasJugadas` ya contiene las jugadas recientes. Observer agregaría suscripción y orden de notificación sin un segundo suscriptor. Se reconsiderará si se agrega un marcador o un registro de eventos.
D6	La interfaz necesita un único punto de entrada que oculte la estructura del dominio.	Facade; acceso directo a `Partida`.	**Facade** en `ControladorJuego`; se documenta en la Entrega 3.	El acceso directo acopla la interfaz a las reglas y obliga a repetir la coordinación de los turnos de las CPU en la consola y en la simulación.
D7	Integrar el reloj del sistema y el almacenamiento en archivos JSON sin que el dominio dependa de ellos.	Adapter; uso directo de `DateTime.Now` y del sistema de archivos.	**Adapter** en `RelojSistema` y `RepositorioPartidasJson`; se documenta en la Entrega 3.	El uso directo impide la simulación con reloj controlado y ata las reglas al medio de almacenamiento.

N.º	Problema y fuerza	Candidatos	Decisión	Por qué se descartaron los otros
D8	Crear los jugadores y su estrategia a partir de la configuración.	Factory Method; Abstract Factory; condicional en `Partida.CrearJugadores` (actual).	**Pendiente** para la Entrega 3.	Hoy un condicional elige la estrategia según el nivel. Se evaluará en la unidad de patrones creacionales, junto con la construcción de la partida.

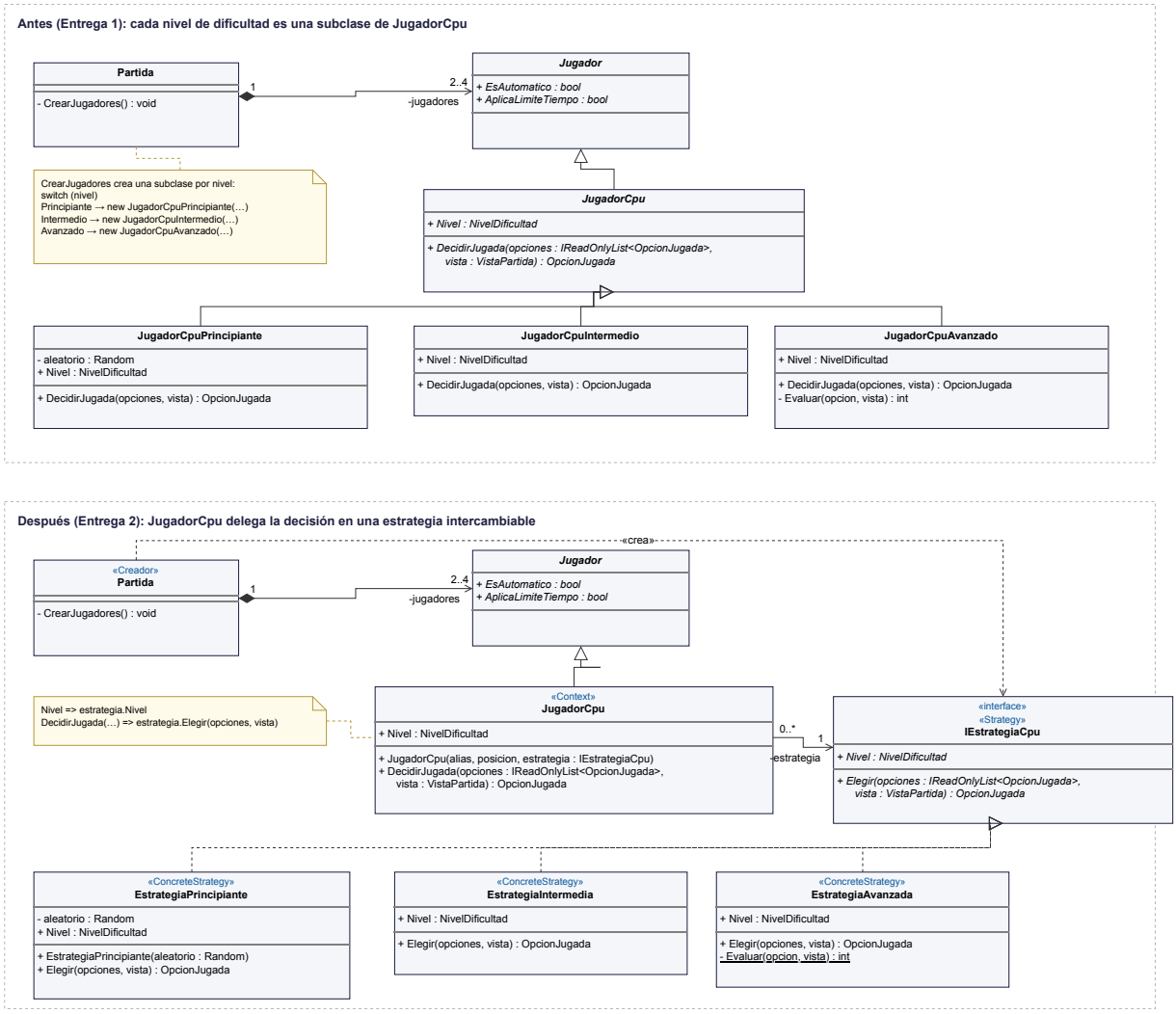
8.2. Ficha del patrón Strategy

Tabla 18
Ficha del patrón Strategy

Campo	Contenido
Patrón y unidad	Strategy. Patrón de comportamiento, Unidad 3.
Problema y fuerzas	Los niveles de CPU solo difieren en cómo eligen entre las opciones válidas: al azar, por más puntos o conservando fichas jugables. En la versión 1.0 cada nivel era una subclase de `JugadorCpu`: el criterio quedaba fijado al crear el jugador y cada nivel nuevo exigía otra subclase. Fuerzas: agregar criterios sin modificar `Partida`, probarlos por separado y decidir sobre `VistaPartida`, sin ver las manos ajenas.
Clases participantes	*Strategy*: `IEstrategiaCpu`. *ConcreteStrategy*: `EstrategiaPrincipiante`, `EstrategiaIntermedia`, `EstrategiaAvanzada`. *Context*: `JugadorCpu`, que delega `DecidirJugada` y `Nivel` en su estrategia. *Cliente*: `Partida.CrearJugadores`.
Estructura	Figura 6.
Consecuencias	**Mejora:** una sola clase de jugador automático; cada criterio es una clase pequeña, sin estado, que se prueba con una vista construida en la prueba; el nivel ya no se duplica, porque lo da la estrategia. **Costo:** una interfaz y una indirección más por decisión, y una clase más en total. Elegir la estrategia según el nivel sigue siendo un condicional en `CrearJugadores`, que se trata en la Entrega 3 (decisión D8).
Alternativa descartada	Template Method: no hay un esqueleto común con pasos que varíen, porque cada nivel reemplaza la decisión completa, y conservaría la herencia que ata el criterio al jugador.
Prueba de que sirve	Un nivel nuevo es una clase que implementa `IEstrategiaCpu` más un caso en `CrearJugadores`, sin tocar `JugadorCpu`, `ResolverTurnosCpu` ni la interfaz; antes exigía una subclase de jugador. En 1000 partidas simuladas con semilla 42 los criterios se distinguen: Avanzado gana 296, Intermedio 252, el humano simulado 229 y Principiante 215 (8 empates). El costo de la indirección se medirá en la Entrega 3.

Figura 6

Strategy: estructura antes y después de aplicarlo



8.3. Ficha del patrón State

Tabla 19
Ficha del patrón State

Campo	Contenido
Patrón y unidad	State. Patrón de comportamiento, Unidad 3.
Problema y fuerzas	Lo que la partida admite depende de su estado: preparada, solo iniciar; salida, solo la ficha de salida y sin tiempo; en curso, colocar, robar, pasar y deshacer con control de tiempo; terminada, nada. En la versión 1.0 el estado era un atributo `EstadoPartida` que cada operación debía consultar con condicionales, como ya lo hacían `Iniciar` y `OpcionesDe`. Fuerzas: las reglas de un estado quedaban repartidas en siete operaciones, un estado nuevo obligaba a revisarlas todas y las transiciones quedaban implícitas.
Clases participantes	*State*: `EstadoJuego`. *ConcreteState*: `EstadoPreparada`, `EstadoSalida`, `EstadoEnCurso`, `EstadoTerminada`. *Context*: `Partida`, que delega en su estado las siete operaciones y ofrece `CambiarEstado`. `EstadoPartida` queda como valor visible en la vista.
Estructura	Figuras 7 (antes y después) y 8 (transiciones).
Consecuencias	**Mejora:** las reglas de cada estado están juntas, las transiciones son explícitas y `Partida` perdió los condicionales; por omisión un estado no admite nada, así que uno nuevo solo habilita lo que declara. **Costo:** cinco clases más; los estados usan operaciones de paquete de `Partida`, lo que los acopla al contexto, y cada acción queda repartida entre la partida y el estado.
Alternativa descartada	Mantener los condicionales de la versión 1.0 y Chain of Responsibility (decisión D2). Tampoco se conservó Cerrada como estado: duraba un instante y no tenía reglas propias; el cierre queda en `Resultado.Forma`.
Prueba de que sirve	La regla de que el tiempo no corre en la salida (RN-04) se expresó solo en los estados, sin tocar `Partida`: `EstadoEnCurso` controla el tiempo y `EstadoSalida` hereda el comportamiento por omisión. Deshacer la salida es una transición a `EstadoSalida`. Las pruebas de `EstadoPruebas` verifican lo que admite cada estado.

Figura 7

State: estructura antes y después de aplicarlo

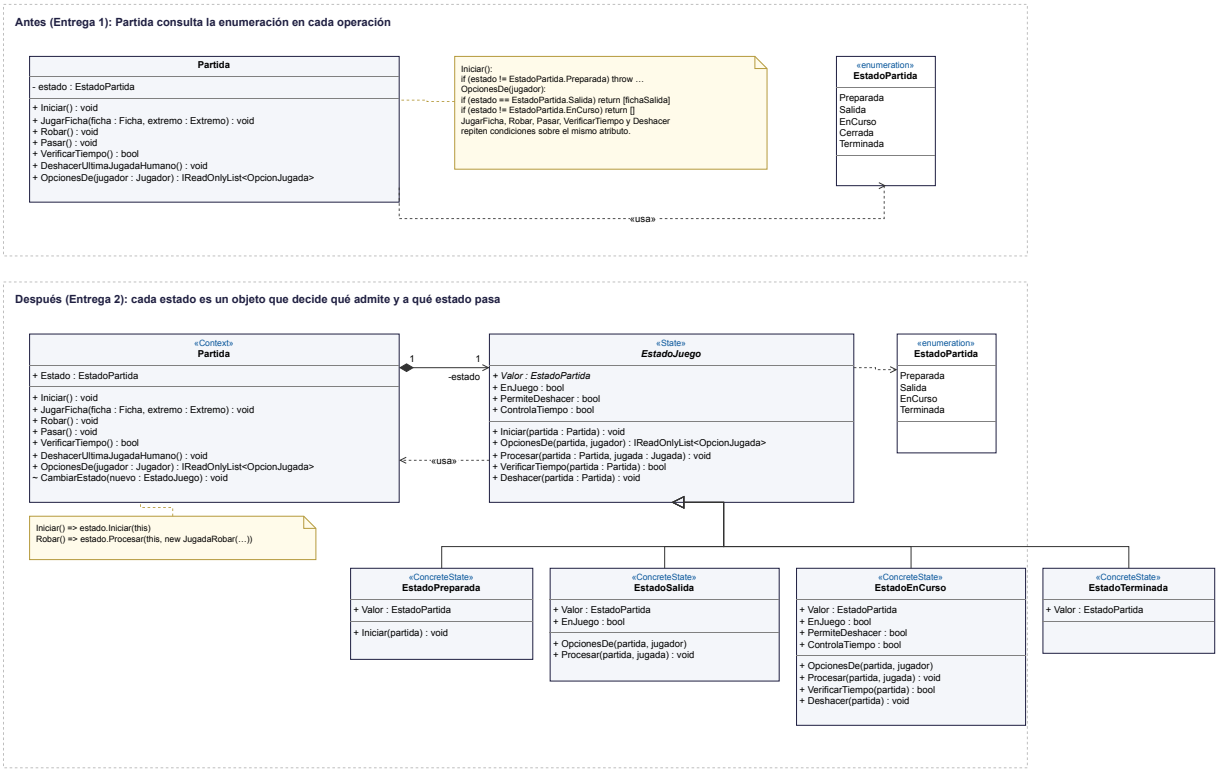
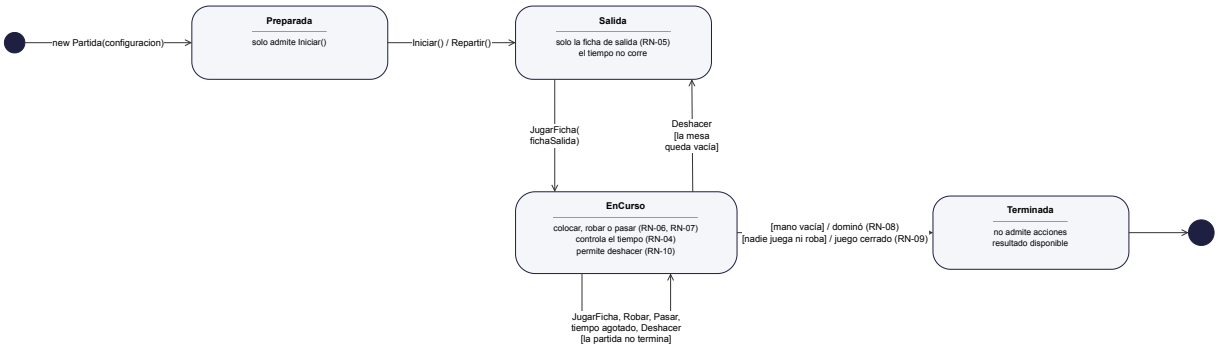


Figura 8

Diagrama de estados de la partida



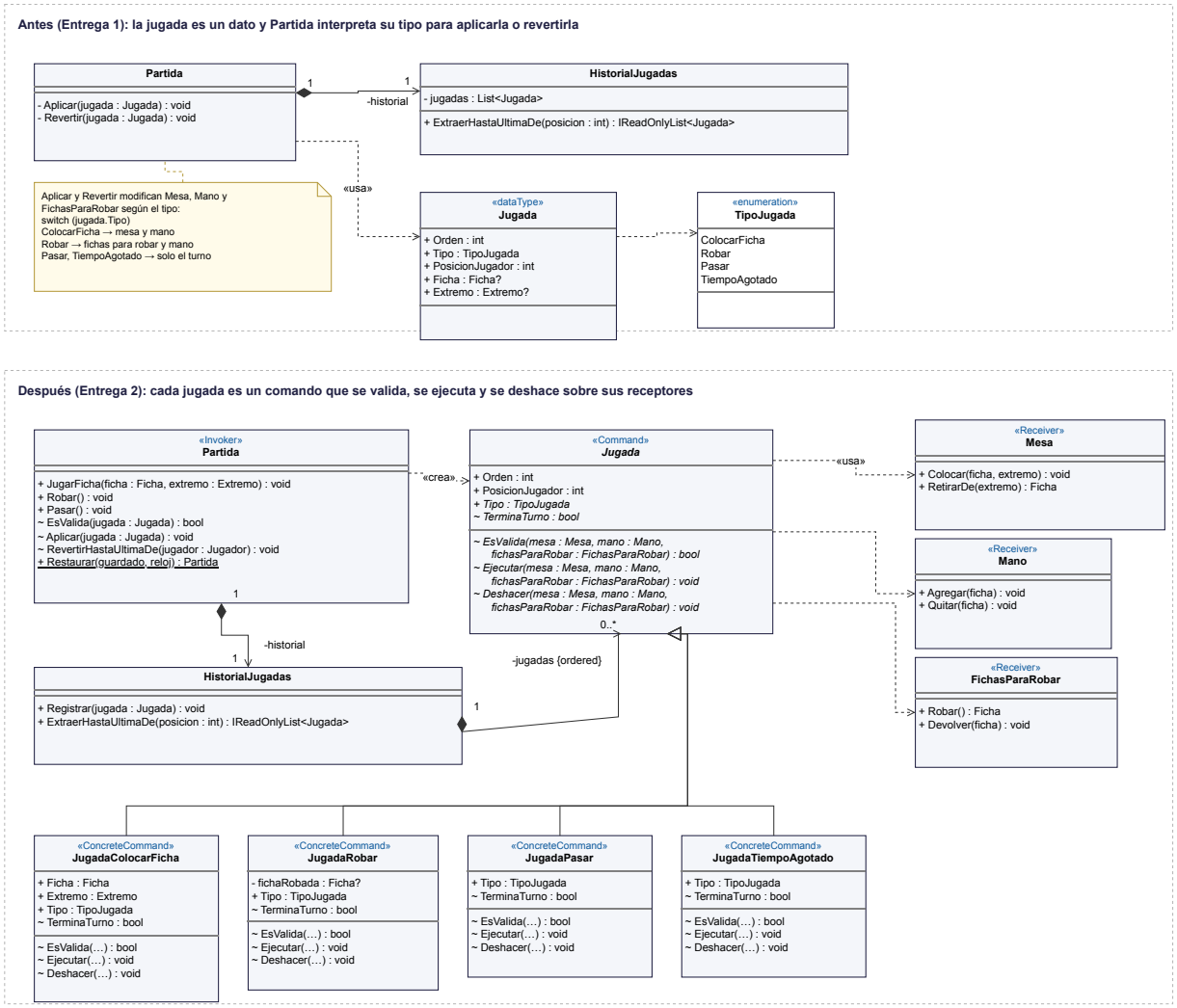
8.4. Ficha del patrón Command

Tabla 20
Ficha del patrón Command

Campo	Contenido
Patrón y unidad	Command. Patrón de comportamiento, Unidad 3.
Problema y fuerzas	Deshacer (RN-10) y guardar y cargar (RN-11) exigen conservar las jugadas en orden, revertirlas y repetirlas. En la versión 1.0 `Jugada` era un dato que `Partida.Aplicar` y `Revertir` debían interpretar con un condicional. Fuerzas: ejecutar y revertir deben ser simétricos; robar exige recordar la ficha; cada tipo nuevo tocaba dos operaciones de `Partida`; el archivo no debe exponer el estado interno.
Clases participantes	*Command*: `Jugada`. *ConcreteCommand*: `JugadaColocarFicha`, `JugadaRobar`, `JugadaPasar`, `JugadaTiempoAgotado`. *Receiver*: `Mesa`, `Mano`, `FichasParaRobar`. *Invoker*: `Partida` (`Aplicar`, `RevertirHastaUltimaDe`, `Restaurar`). `HistorialJugadas` conserva los comandos ejecutados.
Estructura	Figura 9.
Consecuencias	**Mejora:** cada jugada se ejecuta y se deshace sobre sus receptores; la misma secuencia sirve para deshacer, mostrar y guardar; un tipo nuevo no toca `Partida`; la interfaz lee las jugadas sin poder ejecutarlas. **Costo:** cinco clases reemplazan un tipo de valor; `JugadaRobar` no es inmutable; el JSON exige registrar los tipos en el repositorio; cargar vuelve a procesar todas las jugadas, acción que se medirá.
Alternativa descartada	Memento: una instantánea de mesa, manos y fichas antes de cada jugada del humano resolvía deshacer, pero no guardar sin exponer el estado interno ni conservar qué se jugó. Prototype, clonar la partida cada turno, tiene el mismo problema y más memoria (decisión D3).
Prueba de que sirve	Un tipo de jugada nuevo es una subclase de `Jugada` y su registro en el repositorio, sin tocar deshacer, guardar ni `Restaurar`. `JugadaPruebas` verifica que deshacer restaura el estado, incluida la salida, y que cargar reconstruye la partida, también tras deshacer un robo. La simulación deshace, guarda y carga en cada una de sus 1000 partidas.

Figura 9

Command: estructura antes y después de aplicarlo



9. Medición de recursos y tiempo

Las mediciones se realizarán con `dotnet-trace` y `dotnet-counters` sobre el proyecto `Domino.Simulacion`, que ejecuta partidas completas sin interfaz y con semilla fija. En ellas el simulador ocupa el lugar del jugador humano y juega la primera opción válida, de modo que cada partida es reproducible.

9.1. Acciones críticas declaradas

La Tabla 21 presenta las acciones que se medirán y el motivo de su elección.

Tabla 21
Acciones críticas objeto de medición

N.º	Acción crítica	Motivo de la elección
1	Ejecutar jugada	Es la acción más frecuente de la partida.
2	Calcular jugadas válidas	Se invoca en cada turno y la utilizan todos los niveles de CPU.
3	Decidir jugada automática	Se mide por nivel de dificultad; se espera que concentre la mayor parte del costo en el nivel Avanzado.
4	Repartir fichas	Varía según el tipo de dominó y el número de jugadores.
5	Deshacer jugada	Revierte varias jugadas como una unidad.
6	Guardar partida	Involucra escritura en disco.
7	Cargar partida	Involucra lectura de disco y reconstrucción del estado.

9.2. Estado de la medición

En esta entrega `Domino.Simulacion` ejecuta partidas completas con semilla fija y ejercita las siete acciones críticas: en cada partida juega, calcula las jugadas válidas, decide las jugadas de las tres CPU, reparte, deshace, guarda y carga una vez. La corrida de 1000 partidas con semilla 42 es reproducible: dos ejecuciones producen el mismo

resultado. La línea base con `dotnet-trace` y `dotnet-counters` (p50, p95, tiempo de CPU frente a tiempo de reloj y memoria por acción) se presentará en la Entrega 3 sobre este mismo escenario, y servirá para estimar el costo de los patrones aplicados.

10. Bitácora de decisiones

La Tabla 22 registra las decisiones relevantes del proyecto, cada una con la alternativa descartada y la razón que la sustenta.

Tabla 22
Bitácora de decisiones de diseño

Fecha	Decisión	Alternativa descartada	Razón
18/09/2026	Dominó como juego del proyecto, desarrollado en C#.	Otros juegos de mesa.	Ofrece estados, variantes y jugadas reversibles; C# es el lenguaje de mayor dominio de mi parte.
27/09/2026	Partida local de un jugador humano contra una a tres CPU.	Multijugador en red con salas y código de acceso.	La red concentra el esfuerzo en sincronización, desconexiones e información oculta, aspectos que no se evalúan; además consume el presupuesto de clases y dificulta el modo banco de pruebas. La abstracción del jugador conserva la posibilidad de agregar un jugador remoto sin modificar el dominio.
27/09/2026	Juego siempre individual.	Modalidad en parejas.	Las parejas solo aplican con cuatro jugadores y agregan reglas de puntuación compartida sin aportar fuerzas de diseño nuevas.
27/09/2026	Tipos de dominó doble 6 y doble 9.	Doble 12.	No aporta un comportamiento distinto al de doble 9; únicamente prolonga la partida.

Fecha	Decisión	Alternativa descartada	Razón
27/09/2026	Abre la mano quien tenga el doble más alto entre los jugadores (RN-05).	Exigir siempre el doble máximo del juego.	Cuando quedan fichas para robar, el doble máximo puede quedar sin repartir y la mano no podría comenzar.
27/09/2026	Deshacer solo la última jugada del humano, junto con las jugadas posteriores de las CPU.	Deshacer varios niveles y rehacer.	Permite revertir jugadas con menor complejidad; el historial permitiría extenderlo más adelante.
27/09/2026	Partida a una sola mano, sin puntos acumulados.	Varias manos con meta de puntos.	Simplifica el cierre de la partida; los puntos se utilizan únicamente para resolver el juego cerrado.
27/09/2026	Límite de tiempo opcional por turno (10, 20 o 30 s); al agotarse, se pierde el turno.	Que el sistema juegue o robe por el jugador.	Es la regla más simple de comunicar y no decide en nombre del jugador.
27/09/2026	Incluir guardar y cargar partida.	No persistir la partida.	Completa el ciclo de vida de la jugada registrada y permite medir operaciones de disco.
27/09/2026	Acoger el componente de medición.	No presentarlo.	Permite sustentar con datos el costo de las decisiones de diseño.
02/10/2026	Guardar la partida como configuración, semilla y jugadas, y reconstruirla al cargar.	Serializar el estado completo de `Partida`.	No obliga a exponer el estado interno y reutiliza el historial que ya exige deshacer; el costo es repetir las jugadas al cargar, que se medirá como acción crítica.

Fecha	Decisión	Alternativa descartada	Razón
02/10/2026	`Jugada` identifica al jugador por su posición.	Referenciar el objeto `Jugador`.	La jugada es un valor inmutable que se guarda en archivo; la posición la identifica sin acoplarla a la entidad.
02/10/2026	Tipos de valor inmutables (`record`) para fichas, jugadas, resultados, configuración y vista.	Clases mutables relacionadas por asociación.	Se comparan por valor, pueden compartirse sin copias defensivas y la vista no permite modificar el dominio.
02/10/2026	`Partida` recibe `IReloj` y aplica la regla de tiempo.	Calcular el tiempo en `ControladorJuego`.	RN-04 es una regla del juego; ubicarla en el controlador le restaría cohesión.
02/10/2026	Proyecto `Domino.Persistencia` para el repositorio JSON.	Repositorio dentro de `Domino.Consola`.	La simulación también guarda y carga partidas para medir esas acciones.
02/10/2026	Un solo repositorio con el código y el documento.	Repositorios separados.	Una misma etiqueta marca el estado del código y del diseño de cada entrega, y el historial muestra los cambios de ambos juntos.
02/10/2026	En la simulación, el simulador ocupa el lugar del humano.	Partidas solo entre CPU.	La regla RN-02 exige un jugador humano; simularlo conserva la regla y ejercita el mismo código que la consola.
24/10/2026	Strategy para el criterio de decisión de cada nivel de CPU.	Mantener las subclases de `JugadorCpu`; Template Method.	Los niveles solo difieren en la elección completa; la estrategia se agrega sin tocar el jugador ni la partida (decisión D1).

Fecha	Decisión	Alternativa descartada	Razón
24/10/2026	State para las reglas que dependen del estado de la partida.	Condicionales sobre `EstadoPartida`; Chain of Responsibility.	Agrupa las reglas de cada estado y hace explícitas las transiciones (decisión D2).
24/10/2026	Command: `Jugada` pasa de tipo de valor a jerarquía de comandos.	Memento; Prototype.	La misma secuencia de jugadas sirve para deshacer, mostrar y guardar sin exponer el estado interno (decisión D3).
24/10/2026	Retirar el estado Cerrada.	Modelarlo como estado transitorio antes de Terminada.	Duraba un instante y no tenía reglas propias; la forma de cierre ya queda en `Resultado`.
24/10/2026	El límite de tiempo no corre durante la salida (RN-04).	Aplicarlo también a la salida.	Si quien abre pierde el turno, nadie más puede abrir; además, la ficha de salida es una jugada obligada.
24/10/2026	`IniciarPartida` y `CargarPartida` resuelven los turnos de las CPU.	Esperar la primera acción del humano.	Si la salida correspondía a una CPU, la partida quedaba detenida sin opciones para el humano.
24/10/2026	No aplicar Chain of Responsibility a la validación ni Observer a la notificación de jugadas.	Aplicarlos en esta entrega.	No hay una fuerza que los justifique: validaciones fijas y un único interesado en las jugadas (decisiones D4 y D5).
24/10/2026	Registrar los tipos de jugada para el JSON en `Repositorio PartidasJson`.	Atributos de serialización en las clases del dominio.	El dominio no depende del formato del archivo.

Fecha	Decisión	Alternativa descartada	Razón
24/10/2026	Consola interactiva: un solo tablero redibujado, fichas con sus puntos y manejo con teclado, con `PantallaConsola` separada de `InterfazConsola`.	Consola por líneas que se desplaza.	Mejora la jugabilidad sin tocar el dominio; separar el dibujo de la lectura de teclas conserva la cohesión de cada clase.
24/10/2026	El turno se pierde en el instante en que se agota el tiempo: la consola consulta `VerificarTiempo` mientras espera una tecla.	Verificar el tiempo solo al recibir la acción.	Cumple RN-04 sin que el jugador tenga que presionar una tecla para descubrir que perdió el turno.
24/10/2026	Proyecto de pruebas automatizadas `Domino.Pruebas`.	Verificar solo jugando en la consola.	Verifica las reglas y los patrones en cada cambio. No forma parte del diseño ni del conteo de clases.
30/10/2026	Presentar la línea base de medición en la Entrega 3.	Presentarla en esta entrega.	Se priorizaron los patrones obligatorios; la simulación ya ejercita las acciones críticas sobre el escenario que se medirá.

10.1. Declaración de uso de asistentes de inteligencia artificial

En la Entrega 1 se utilizó un asistente de inteligencia artificial para explorar alternativas de alcance del juego, revisar la redacción del caso de uso CU-01 frente a la plantilla del curso, proponer borradores del modelo de dominio y del diagrama de clases, mejorar la redacción de las secciones de este documento, organizar su estructura en LaTeX y revisar el diagrama de clases frente a las reglas de negocio. Las decisiones registradas en esta bitácora, incluidas las alternativas descartadas, fueron de análisis propio.

En la Entrega 2 se utilizó un asistente de inteligencia artificial para analizar la pertinencia de los patrones escogidos frente al juego y como apoyo en la escritura del documento.

Referencias bibliográficas

Cockburn, A. (2001). *Writing Effective Use Cases*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. E., and Vlissides, J. (2003). *Patrones de diseño: elementos de software orientado a objetos reutilizable*. Addison-Wesley.

Kimmel, P. (2008). *Manual de UML*. McGraw-Hill Interamericana.

Larman, C. (2003). *UML y patrones: una introducción al análisis y diseño orientado a objetos y al proceso unificado*. Pearson, 2 edition.

Martínez Juan, F. J. (2011). *Construcción de software Java con patrones de diseño*. Universidad de Oviedo.